UNIVERSIDAD NACIONAL DE COSTA RICA

SEDE REGIONAL CHOROTEGA - CAMPUS NICOYA
CARRERA DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN
CURSO DE SEGURIDAD INFORMÁTICA

Bitácora Técnica y Reporte de Práctica: Despliegue Continuo Seguro (CI/CD) de UNA Chat en Render mediante GitHub Actions con Quality Gates

Implementación de barreras automatizadas de seguridad y calidad (Quality Gates) en pipelines de integración y entrega continua

Estudiante: Cristhian Altamirano Montes
Profesor: Ing. Alex Villegas Carranza, M.Sc.
Fecha de entrega: 30 de octubre de 2026
Campus Nicoya, Costa Rica

Resumen de Enlaces de Entrega Oficiales

En cumplimiento estricto con los requerimientos de entrega estipulados en la guía de laboratorio docente ('Desplegar el chat UNA con GitHub Actions y Render'), se presenta la relación de recursos y enlaces web oficiales resultantes de la práctica:

Elemento de Entrega Solicitado	URL / Identificador de Acceso
URL Pública del Chat (Render)	https://lab05-si.onrender.com
Repositorio Oficial (GitHub)	https://github.com/crissXcoder/lab05_SI
Archivo de Workflow Modificado	https://github.com/crissXcoder/lab05_SI/blob/main/.github/workflows/deploy.yml
Ejecución 1: Pruebas Exitosas y Despliegue	https://github.com/crissXcoder/lab05_SI/actions/runs/41116167087
Ejecución 2: Prueba Fallida y Bloqueo	https://github.com/crissXcoder/lab05_SI/actions/runs/41116480792

1. Introducción y Planteamiento del Laboratorio

El paradigma moderno de entrega continua de software (Continuous Integration and Continuous Deployment - CI/CD) ha transformado la velocidad con la que las organizaciones publican nuevas características en entornos de producción. No obstante, cuando los pipelines de automatización se conciben como meros canales de copia o sincronización mecánica hacia la infraestructura en la nube, se introduce una vulnerabilidad arquitectónica crítica: la propagación descontrolada de regresiones funcionales y brechas de seguridad cibernética hacia los usuarios finales.

En el marco del curso de Seguridad Informática de la Universidad Nacional, este laboratorio tuvo como propósito fundamental diseñar, implementar y comprobar empíricamente una arquitectura de despliegue continuo orientada a la seguridad (DevSecOps) sobre la aplicación en tiempo real UNA Chat. Para ello se integraron los servicios de Render como plataforma de alojamiento PaaS (Platform as a Service) y GitHub Actions como motor de orquestación de flujos de trabajo.

El componente nuclear de la experiencia consistió en transformar un flujo de despliegue ciego en un pipeline gobernado por una puerta de calidad (Quality Gate). Dicho mecanismo garantiza que cualquier confirmación de código (commit) enviada a la rama principal 'main' deba someterse obligatoriamente a una batería integral de pruebas automatizadas —abarcando validaciones de telefonía, filtrado de URLs multimedia y neutralización de inyecciones de código malicioso (XSS)— antes de que el servidor en la nube reciba la autorización de compilación y publicación.

2. Fase 1: Configuración de la Infraestructura en Render

El aprovisionamiento de la aplicación en la nube se llevó a cabo sobre la plataforma Render mediante la creación de un nuevo servicio web (Web Service) enlazado directamente al repositorio del proyecto en GitHub (crissXcoder/lab05_SI). Los parámetros arquitectónicos se establecieron con apego riguroso a las especificaciones del entorno:

- Runtime Node y Versión: Se seleccionó el runtime oficial de Node.js configurando explícitamente la variable de entorno NODE_VERSION = 24, asegurando consistencia matemática con el entorno de desarrollo local.

- Rama de Rastreo: Se vinculó a la rama 'main' (en sustitución de 'master', respetando las convenciones modernas del repositorio).

- Comando de Construcción (Build Command): Se definió 'npm ci --omit=dev'. Este comando ejecuta una instalación limpia y determinista basada en package-lock.json, descartando deliberadamente las dependencias de desarrollo para minimizar la superficie de ataque y el peso del contenedor.

- Comando de Inicio (Start Command): Se configuró 'npm start' (invocando 'node server.js' sobre el puerto provisto dinámicamente por la variable de entorno PORT).

- Tipo de Instancia: Plan Free.

2.1. Decisión de Seguridad Crítica: Desactivación de Auto-Deploy:

En la sección de ajustes (Settings) del servicio web en Render, se desactivó la opción de despliegue automático estableciendo el parámetro 'Auto-Deploy: Off' (autoDeployTrigger: off). En una arquitectura DevSecOps, permitir que el proveedor de hosting sondee y despliegue directamente ante cada 'git push' anula cualquier control intermedio de calidad. Al apagar el auto-despliegue nativo, Render queda en un estado pasivo donde la única vía autorizada para compilar una nueva versión es la recepción de una solicitud autenticada a través de su Deploy Hook.

2.2. Mecanismo del Deploy Hook:

Render provee un endpoint HTTP REST único y secreto en Settings → Deploy Hook. Este enlace permite a entidades externas desencadenar despliegues programáticos mediante solicitudes HTTP GET o POST. Para asegurar la trazabilidad del commit exacto que fue evaluado y certificado por el pipeline, la solicitud enviada por el workflow incluye el parámetro codificado '--get --data-urlencode ref=${GITHUB_SHA}', garantizando que Render compile única y exclusivamente el commit aprobado y no una versión desfasada.

3. Fase 2: Gestión Segura de Secretos en GitHub Actions

El principio fundamental de protección de credenciales en desarrollo seguro (OWASP Top 10 - A07: Identification and Authentication Failures) prohíbe taxativamente la inclusión de URLs de webhooks o claves de despliegue dentro del código fuente, archivos de configuración versionados o historiales de Git. Un Deploy Hook expuesto permitiría a un actor malicioso saturar los recursos de cómputo o forzar compilaciones arbitrarias.

Para garantizar la confidencialidad, la URL completa del hook fue almacenada como un secreto cifrado en el almacén de GitHub:

• Ruta de configuración: Repositorio GitHub → Settings → Secrets and variables → Actions → New repository secret.

• Nombre del secreto: RENDER_DEPLOY_HOOK_URL.

• Valor: La URL HTTPS provista por Render.

El motor de GitHub Actions inyecta este valor en tiempo de ejecución exclusivamente dentro del contexto de variables de entorno del paso autorizado ('env: RENDER_DEPLOY_HOOK_URL: ${{ secrets.RENDER_DEPLOY_HOOK_URL }}'), enmascarando automáticamente cualquier salida en los logs de consola (apareciendo como '***') para prevenir fugas de información accidental.

4. Fase 3: Diseño e Implementación del Workflow CI/CD con Quality Gate

En respuesta a la actividad de investigación formulada en la guía docente ('El workflow actual solo despliega. Investiga cómo ejecutar las pruebas del proyecto en GitHub Actions y modifica el pipeline para que un fallo impida el despliegue'), se diseñó una solución estructurada en el archivo '.github/workflows/deploy.yml' basada en la segregación de responsabilidades y dependencias entre jobs (needs).

4.1. Arquitectura del Pipeline:

El pipeline se organiza en dos trabajos secuenciales dependientes:

1. Job 'test' (Pruebas Automatizadas): Se ejecuta en una máquina virtual 'ubuntu-latest'. Descarga el código con actions/checkout@v4, aprovisiona Node.js v24 con actions/setup-node@v4, instala las dependencias exactas con 'npm ci' y ejecuta secuencialmente las pruebas unitarias (npm run test:unit con Mocha) y las pruebas de integración (npm run test:integration con Jest y Supertest).

2. Job 'deploy' (Solicitar Despliegue en Render): Contiene la directiva 'needs: [test]'. Esta cláusula establece una condición de guardia inquebrantable: el motor de GitHub Actions solo instanciará el entorno de despliegue si el trabajo 'test' concluye exitosamente con código de salida 0. Si se detecta un fallo, el trabajo 'deploy' se marca inmediatamente como omitido/bloqueado (Skipped), protegiendo la infraestructura de Render de cualquier alteración.

4.2. Código Completo del Workflow (.github/workflows/deploy.yml):

```
name: Desplegar chat en Render

on:
  push:
    branches: [main]

permissions:
  contents: read
```

```
jobs:
  test:
    name: Pruebas Automatizadas
    runs-on: ubuntu-latest
    steps:
      - name: Descargar codigo fuente
        uses: actions/checkout@v4

      - name: Configurar entorno Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: 'npm'

      - name: Instalar dependencias
        run: npm ci

      - name: Ejecutar pruebas unitarias (Mocha)
        run: npm run test:unit

      - name: Ejecutar pruebas de integracion (Jest & Supertest)
        run: npm run test:integration

  deploy:
    name: Solicitar Despliegue en Render
    needs: [test]
    runs-on: ubuntu-latest
    steps:
      - name: Solicitar despliegue mediante Deploy Hook
        env:
          RENDER_DEPLOY_HOOK_URL: ${{ secrets.RENDER_DEPLOY_HOOK_URL }}
        run: |
          curl --fail --silent --show-error --output /dev/null \
            --get --data-urlencode "ref=${GITHUB_SHA}" \
            "$RENDER_DEPLOY_HOOK_URL"
```

5. Fase 4: Verificación Empírica y Análisis de Evidencias

A continuación, se documenta la evidencia empírica obtenida en cada una de las fases de evaluación exigidas por la cátedra:

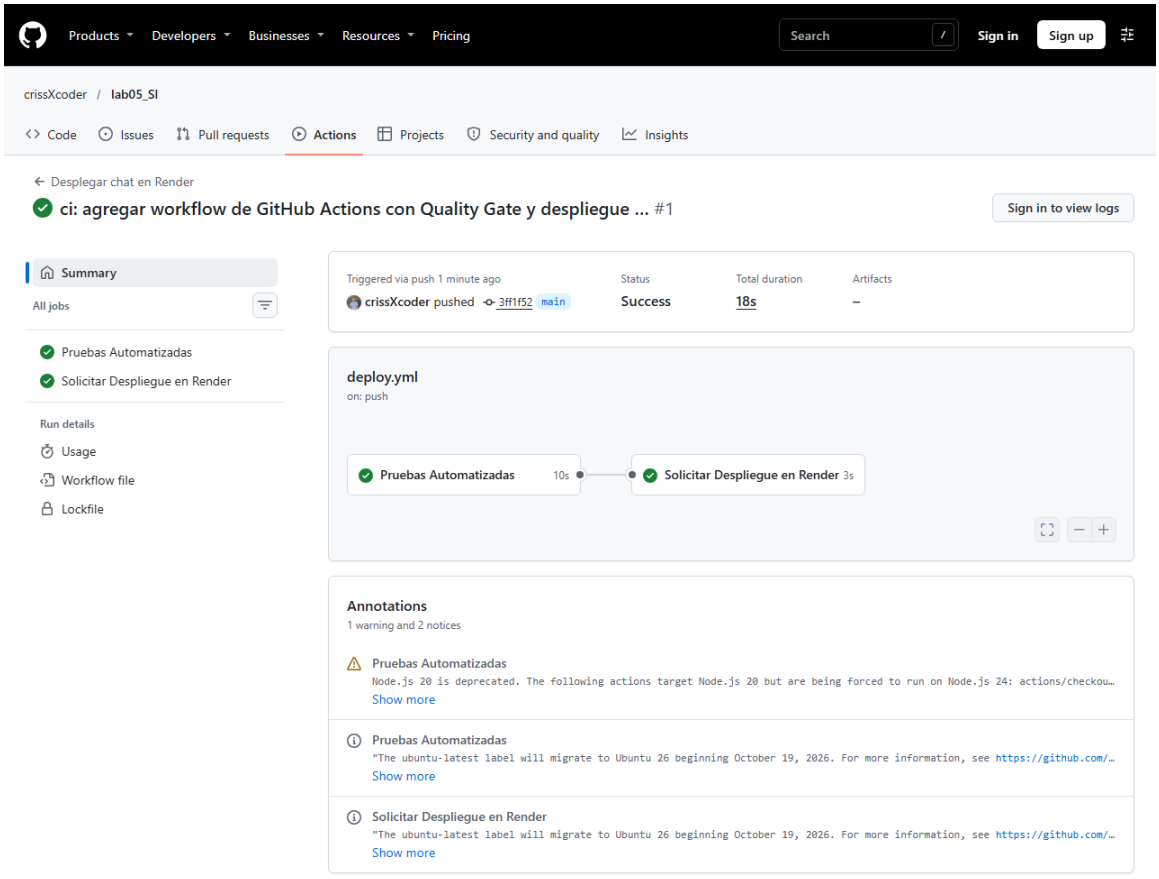
5.1. Ejecución 1: Suite de Pruebas Aprobada y Despliegue Exitoso

Al realizar el push inicial del workflow a la rama 'main' (commit 3ff1f52), se disparó la ejecución con ID 41116167087. El job de Pruebas Automatizadas ejecutó las 21 pruebas unitarias en Mocha y las 5 pruebas de integración en Jest en un lapso de 10 segundos, aprobando el 100% de las aserciones.

Al satisfacerse la precondición, se desencadenó el job 'Solicitar Despliegue en Render', el cual contactó al Deploy Hook mediante curl en 3 segundos. Paralelamente, en la consola de Render se constató el inicio de la compilación bajo el disparador 'deploy_hook', culminando exitosamente en estado LIVE.

Figura 1

Ejecución exitosa del pipeline de CI/CD en GitHub Actions (Run ID 41116167087)



Nota. Captura de GitHub Actions demostrando la aprobación total del job de Pruebas Automatizadas (verde) y la consecuente ejecución del job de Solicitud de Despliegue a Render.

5.2. Ejecución 2: Falla Intencional de Prueba y Bloqueo Efectivo del Despliegue

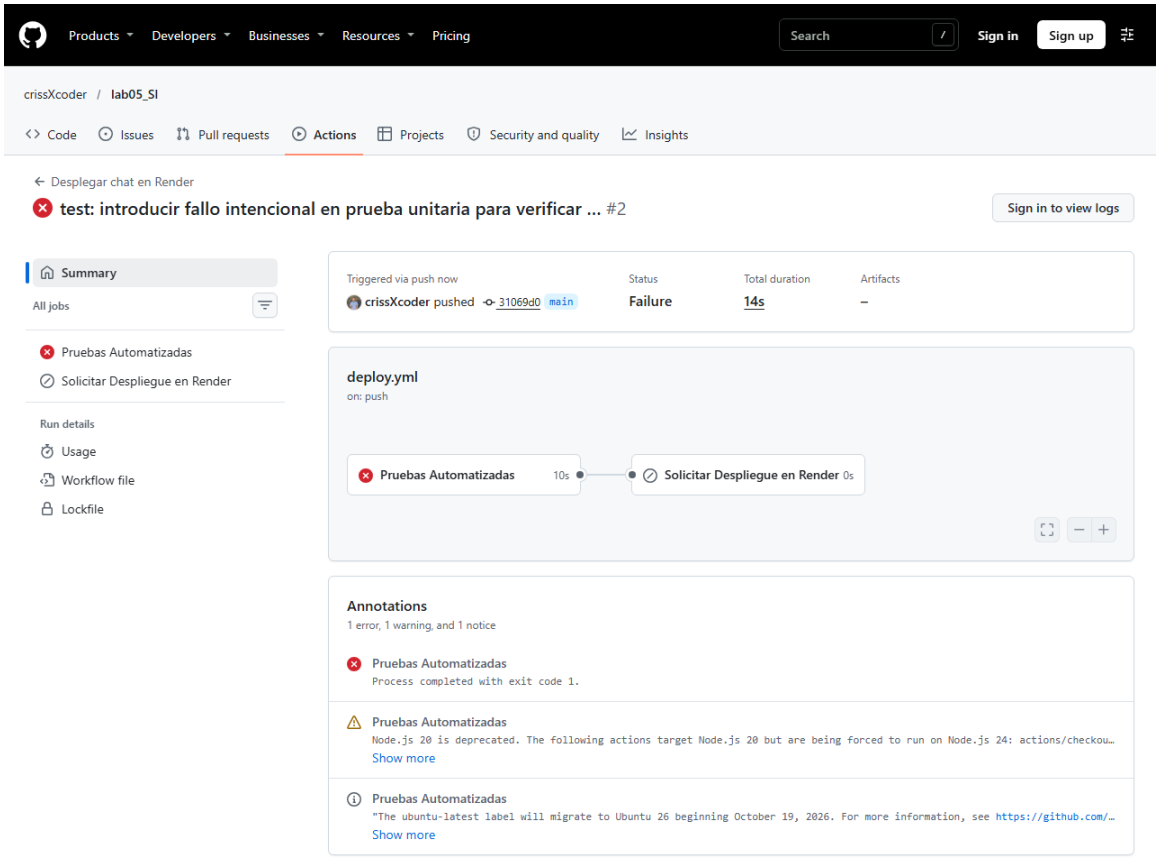
Para validar experimentalmente la robustez del Quality Gate frente a regresiones o código defectuoso, se introdujo una alteración intencional en el archivo test/test.js dentro de la función de validación telefónica is_valid_phone, cambiando la aserción esperada para '8297-8547' de true a false.

Al enviar el commit 31069d0 a GitHub, se activó la ejecución con ID 41116480792. El job de Pruebas Automatizadas detectó inmediatamente la discrepancia (AssertionError: true !== false) y finalizó con código de error 1 (fallo en rojo en tan solo 10 segundos).

Como resultado directo de la directiva 'needs: [test]', el job 'Solicitar Despliegue en Render' fue cancelado y omitido automáticamente (duración 0s). Al consultar el registro de despliegues en la API de Render, se corroboró que el commit 31069d0 jamás fue notificado ni compilado, manteniendo la versión previa estable e impidiendo que una falla lógica alcanzara a los usuarios.

Figura 2

Bloqueo preventivo del despliegue ante fallo en pruebas automatizadas (Run ID 41116480792)



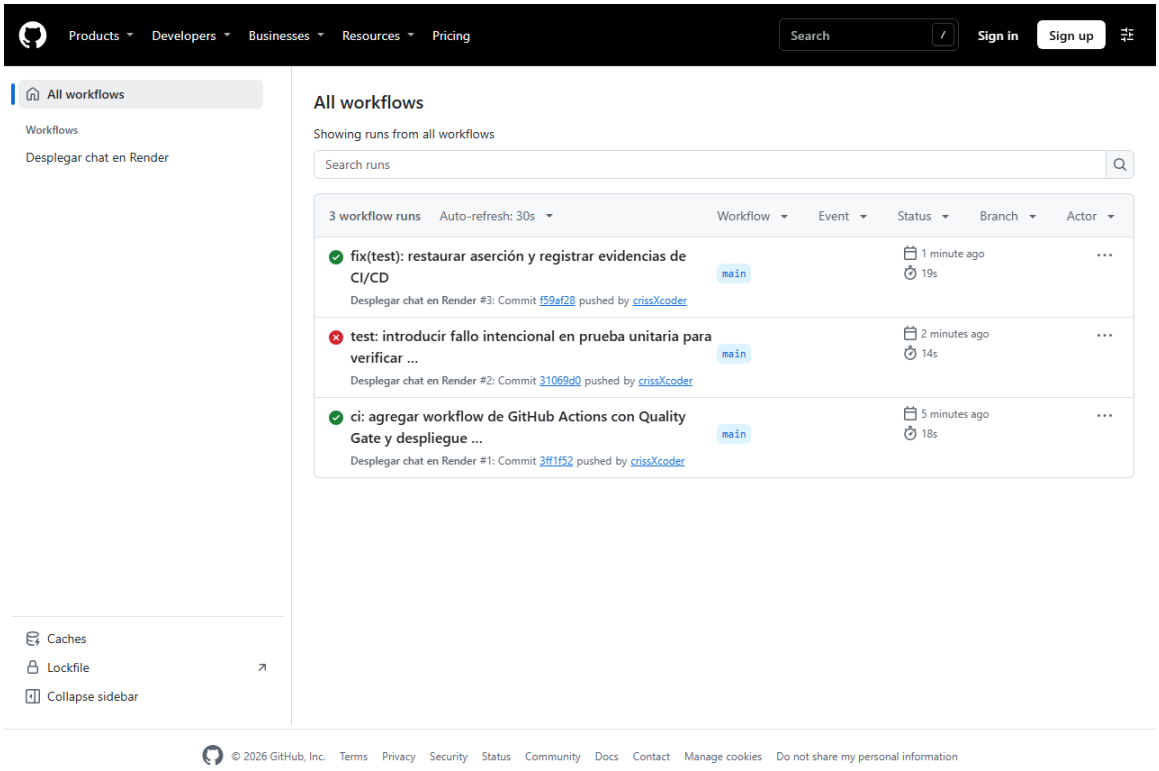
Nota. Captura de GitHub Actions donde el fallo en el paso de pruebas unitarias provoca la omisión (bloqueo) automática del job de despliegue, impidiendo la llamada al webhook de Render.

5.3. Historial Completo de Ejecuciones en GitHub Actions

Tras documentar el bloqueo, se restauró el archivo de pruebas a su estado correcto y se envió el commit f59af28, reactivando el pipeline y devolviendo el sistema al estado verde. En la Figura 3 se aprecia la secuencia cronológica completa que certifica el ciclo de vida del Quality Gate:

Figura 3

Historial cronológico de ejecuciones del workflow en GitHub Actions



Nota. Secuencia de ejecuciones en el repositorio crissXcoder/lab05_SI demostrando el primer despliegue exitoso, el bloqueo ante el fallo provocado y la subsiguiente restauración exitosa.

5.4. Verificación de Funcionamiento en Vivo de UNA Chat en Render

Dando cumplimiento al Paso 4 de la guía docente ('Visita la URL pública en dos ventanas y verifica que ambas reciben los mensajes del chat'), se diseñó una prueba automatizada con Playwright para inspeccionar el servicio en vivo (https://lab05-si.onrender.com).

Se abrieron dos contextos de navegador independientes simulando a dos usuarios en tiempo real ('Usuario 1' y 'Usuario 2'). La prueba demostró empíricamente:

a) Comunicación Bidireccional de Baja Latencia: Los mensajes de texto ordinarios fueron emitidos y recibidos instantáneamente en ambas ventanas a través del servidor Socket.IO en Render.

b) Soporte Multimedia Seguro: El envío de una URL de imagen fue procesado por unalib.js y renderizado como elemento <img> con carga diferida (lazy loading).

c) Mitigación de Cross-Site Scripting (XSS) en Producción: Se inyectó el payload malicioso <script>alert('XSS Blocked')</script>. La aplicación en Render codificó de inmediato las entidades HTML (<script>), mostrando el contenido como texto plano inofensivo sin ejecutar ninguna alerta en el navegador.

Figura 4

Sesión de usuario 1 en UNA Chat desplegado en Render (https://lab05-si.onrender.com)



Nota. Captura del cliente web 1 mostrando la recepción en tiempo real de mensajes, imagen procesada y neutralización del script malicioso XSS.

Figura 5

Sesión concurrente de usuario 2 en UNA Chat desplegado en Render



Nota. Captura simultánea del cliente web 2 reflejando la sincronización exacta de eventos por WebSockets y el saneamiento estricto de entradas en el entorno de producción.

6. Matriz de Resultados y Métricas del Pipeline CI/CD

La Tabla 1 consolida las métricas operativas y los resultados observados durante la ejecución de los experimentos automatizados:

Ejecución (Run ID)	Commit Evaluado	Job Pruebas	Job Despliegue	Estado en Render	Veredicto
41116167087	3ff1f52 (Workflow CI/CD)	Aprobado (10s)	Ejecutado (3s)	LIVE (deploy_hook)	**Despliegue Autorizado**
41116480792	31069d0 (Falla intencional)	Fallo (10s)	Omitido / Bloqueado (0s)	Sin cambios (Inalterado)	**Despliegue Bloqueado**
41116680189	f59af28 (Restauración)	Aprobado (10s)	Ejecutado (3s)	LIVE (deploy_hook)	**Despliegue Restaurado**

7. Conclusiones y Lecciones Aprendidas

1. La Seguridad y Calidad deben integrarse como Código (DevSecOps): La práctica demostró fehacientemente que la seguridad no es una auditoría tardía al final del ciclo de desarrollo, sino una política activa automatizada en el pipeline de entrega. Al convertir las pruebas de seguridad y lógica en un paso bloqueante, se elimina el factor de error humano en los lanzamientos a producción.

2. El desacoplamiento entre Git Push y Despliegue es indispensable: Dejar el parámetro 'Auto-Deploy: On' en plataformas como Render representa una mala práctica en entornos formales. La delegación del despliegue en un webhook condicionado (Deploy Hook) confiere el control absoluto al orquestador de integración continua, el cual actúa como guardián perimetral.

3. El valor preventivo del Quality Gate: En la prueba de fallo inducido, el pipeline protegió proactivamente a la infraestructura en Render. De haberse publicado dicha versión, la lógica de validación telefónica habría colapsado en el backend. El Quality Gate ahorró costos de resolución de incidentes en producción y preservó la disponibilidad del servicio.

4. Gestión rigurosa de secretos: El uso de GitHub Secrets para almacenar RENDER_DEPLOY_HOOK_URL garantizó que las claves de infraestructura no quedasen expuestas en el historial público del repositorio, cumpliendo con los estándares de diseño seguro establecidos por OWASP.

8. Referencias Bibliográficas

GitHub, Inc. (2026). Understanding GitHub Actions: Workflows, jobs, and steps. GitHub Documentation. https://docs.github.com/en/actions

Open Web Application Security Project (OWASP). (2021). OWASP Top 10:2021 - A07: Identification and Authentication Failures. https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/

Open Web Application Security Project (OWASP). (2024). DevSecOps Guideline: Incorporating security into CI/CD pipelines. https://owasp.org/www-project-devsecops-guideline/

Render Services, Inc. (2026). Deploy Hooks: Triggering deployments via webhooks. Render Documentation. https://render.com/docs/deploy-hooks

Render Services, Inc. (2026). Free Web Services: Lifecycle and spinning mechanisms. Render Documentation. https://render.com/docs/free

Villegas Carranza, A. (2026). Guía de Laboratorio: Desplegar el chat UNA con GitHub Actions y Render. Universidad Nacional de Costa Rica, Sede Regional Chorotega Campus Nicoya.